

DL for ENG

LECTURE 5.2

RNN, LSTM and Attention

Networks for data in time: carry a memory from step to step, or read every step directly.

Remus Teodorescu

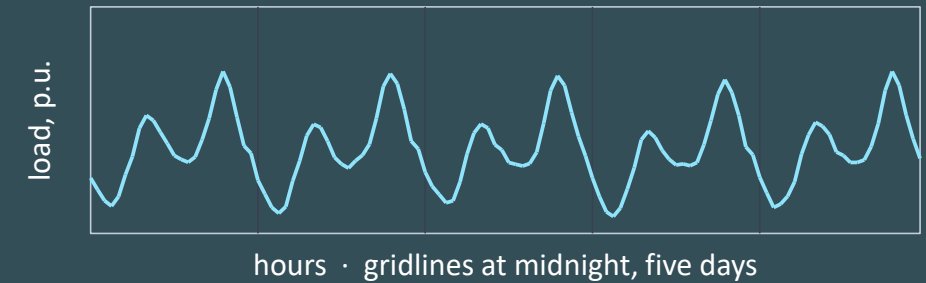
With support from Research Assistant Noman Khan - nomank@energy.aau.dk

Aalborg University · ret@et.aau.dk

Deep Learning for Engineering · 2026

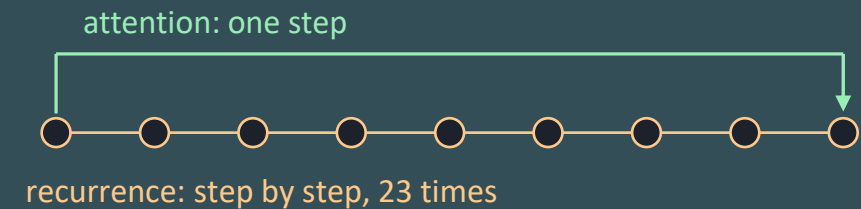
the RNN · the LSTM · attention · one transformer layer

FIVE DAYS OF SUBSTATION DEMAND



notebook 04 · the next hour from the last 24

TWO ROUTES FROM HOUR 1 TO HOUR 24



Learning Objectives

SECTIONS, IN ORDER

- 1 **Sequences** — data in time, and the baseline to beat
- 2 **RNN** — a network with a memory, and why it forgets
- 3 **LSTM** — a memory that lasts, and how to use it
- 4 **Attention** — reading every step directly, and one transformer layer
- 5 **Comparing the models** — notebook 04's four, measured
- 6 **Challenges** — what goes wrong first, and what to check

WHAT WE SKIP, AND WHERE IT IS TAUGHT

training gradients and backpropagation — L6.1

post-training instruction tuning, RL — L6.2

*RNN and LSTM: GBC ch. 10 · attention and
transformers: UDL ch. 12*

Recap

1 – 5 · CONVOLUTION AND RESIDUAL

- 1** What kind of network is a convolutional network, and what function does it implement? *UDL ch. 10.1 – 10.3*
- 2** Describe the architecture of a convolutional network, layer by layer: what happens to resolution and channels? *UDL ch. 10.3 – 10.4; Ex_05 notebook 01*
- 3** How does a convolutional network differ from a dense network in connections, weights, parameters and symmetry? *UDL ch. 10.2; Ex_05 notebook 01*
- 4** Given those differences, what are convolutional networks used for, and which task needs equivariance, not invariance? *UDL ch. 10.1, 10.5*
- 5** Why does a residual connection help the gradient, and what does a block with nothing to add learn? *UDL ch. 11.1 – 11.2*

6 – 10 · GRAPHS

- 6** What kind of network is a graph network, and what function does it implement on node features and adjacency? *UDL ch. 13.1 – 13.3*
- 7** Describe the architecture of a graph network: the three steps of a layer, the readout, and K against the diameter? *UDL ch. 13.4, 13.7; Ex_05 notebook 02*
- 8** How does a graph network differ from a dense network, and what does permutation equivariance guarantee? *UDL ch. 13.2; Ex_05 notebook 03*
- 9** Given those differences, what are graph networks used for, and what is the difference between inductive and transductive? *UDL ch. 13.3, 13.6*
- 10** When is a graph network the wrong choice, and what did notebook 03 measure that shows it? *Ex_05 notebook 03*

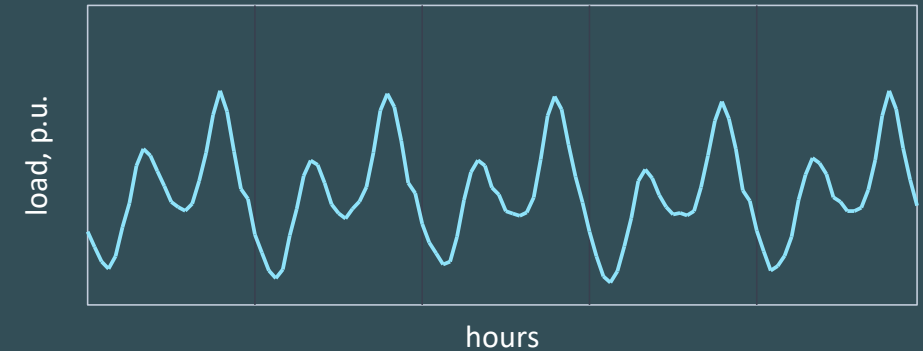
Sequences

DATA IN TIME

- **examples** — grid demand, a sensor trace, a sentence
- **order matters** — shuffle the hours and the pattern is gone
- **the task in notebook 04** — the next hour from the last 24
- **persistence** — predict that the next hour equals this one
- **the baseline to beat** — any model must do better than that

TALK · 2 MIN How do you understand a sequence — what makes one different from a table of rows in any order? Then, in the five days of demand on the right, which past hours would you use to predict the next one?

FIVE DAYS OF DEMAND



WHAT A SEQUENCE MODEL NEEDS

read in order	one step after another
remember	what came before

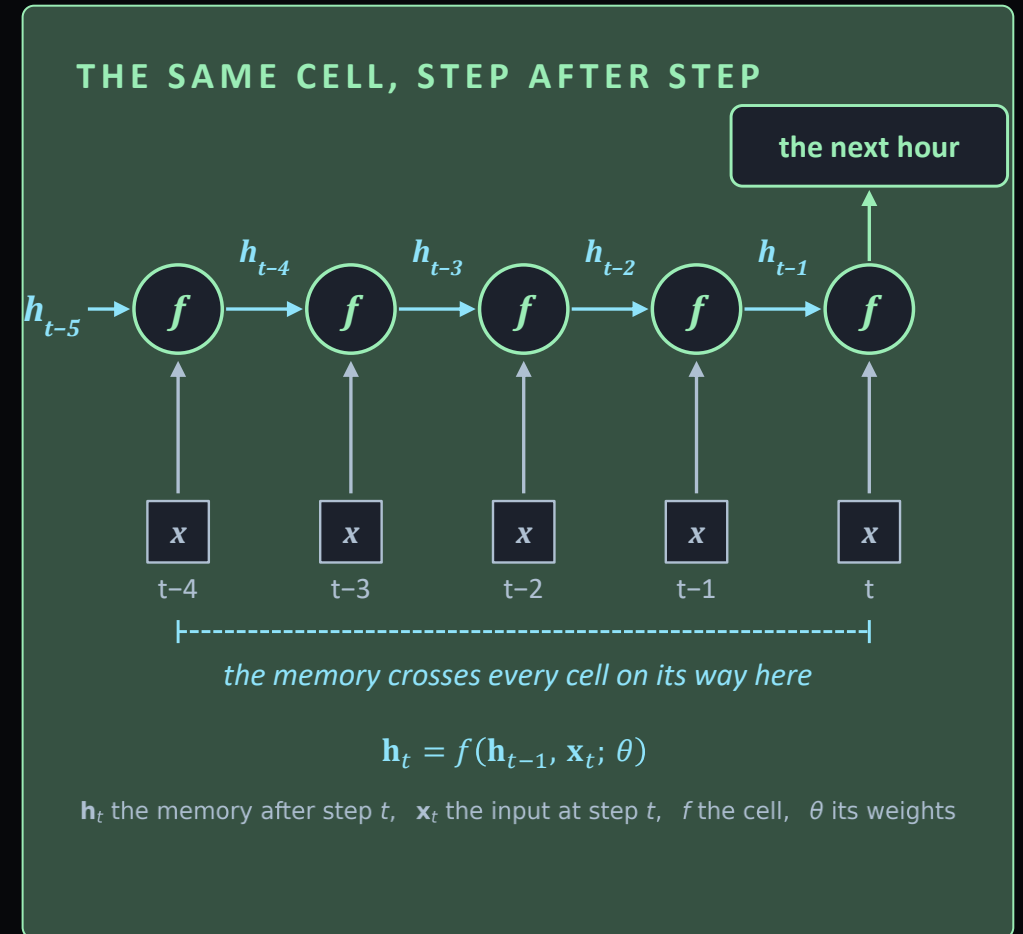
Recurrent Network

KIND, FUNCTION, ARCHITECTURE, APPLICATIONS

- **kind** — a network with a memory, fed back at every step
- **function** — a sequence in, one answer at the end
- **architecture** — one cell, applied at every step
- **applications** — load and wind forecasting, control signals, speech

AGAINST A DENSE NETWORK

	dense network	recurrent network
input	one vector, fixed length	a sequence of any length
memory	none between inputs	the hidden state h
weights	one set per input position	one cell, shared over time
parameters	grow with the window	independent of the length
order	an ordinary input index	built in: step t follows $t - 1$



Hidden State

A SUMMARY OF THE PAST, UPDATED EACH STEP

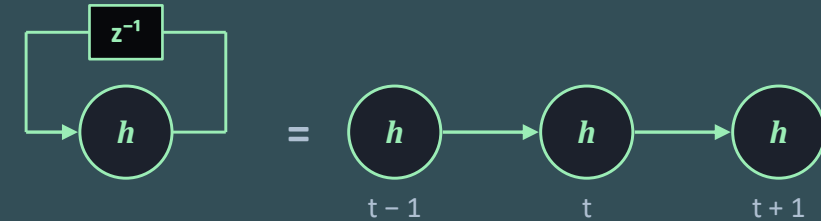
- **the hidden state** — a short summary of everything seen so far
- **each step** — combine the new input with the state, apply tanh
- **the same weights every step** — so any length is fine
- **to train, unfold** — one copy per step, no new parameters

$$\mathbf{h}_t = \tanh(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b})$$

$$\hat{y} = \mathbf{W}_y \mathbf{h}_T + b_y$$

$\mathbf{x}_t$ the input at step t , $\mathbf{h}_t$ the state, $\mathbf{W}_h, \mathbf{W}_x$ shared every step, $\hat{y}$ read from the last state

FOLDED AND UNFOLDED



z^{-1} in the black box: one step of delay

IN NOTEBOOK 04, AS A LOOP

```
h = h0
for t in 1 ... T:
    h = tanh(Wx @ x[t] + Wh @ h + b)
    y = Wy @ h + by
```

W_x reads the input, W_h reads the memory — two jobs, two matrices, both the same at every step

Vanishing Gradient

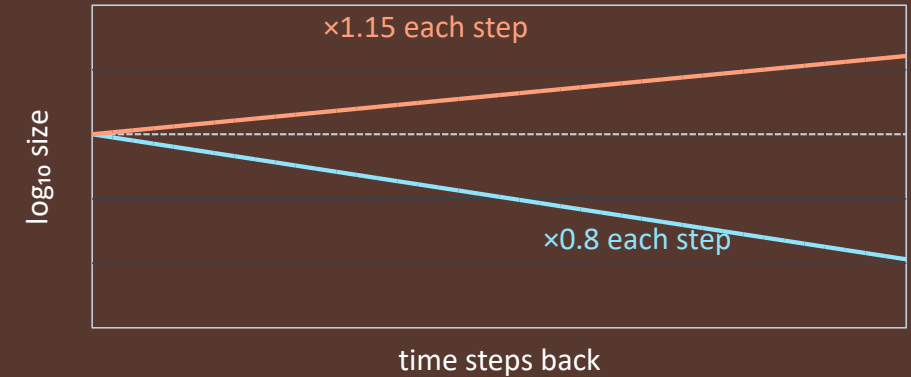
THE SAME PROBLEM AS DEPTH, ALONG TIME

- **back through time** — one factor per step, multiplied together
- **24 steps** — as deep as a 24-layer network
- **worse than depth** — the same matrix, again and again
- **below one it vanishes** — above one it explodes
- **exploding has a fix** — clipping; vanishing does not

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_T} \prod_{t=2}^T \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}}$$

$\mathcal{L}$ the loss, $\mathbf{h}_1$ the first state, T the sequence length, the product is what vanishes

GRADIENT SIZE, BACK IN TIME



GRADIENT CLIPPING

exploding	rescale when too large: fixed
vanishing	the information is lost: not fixed

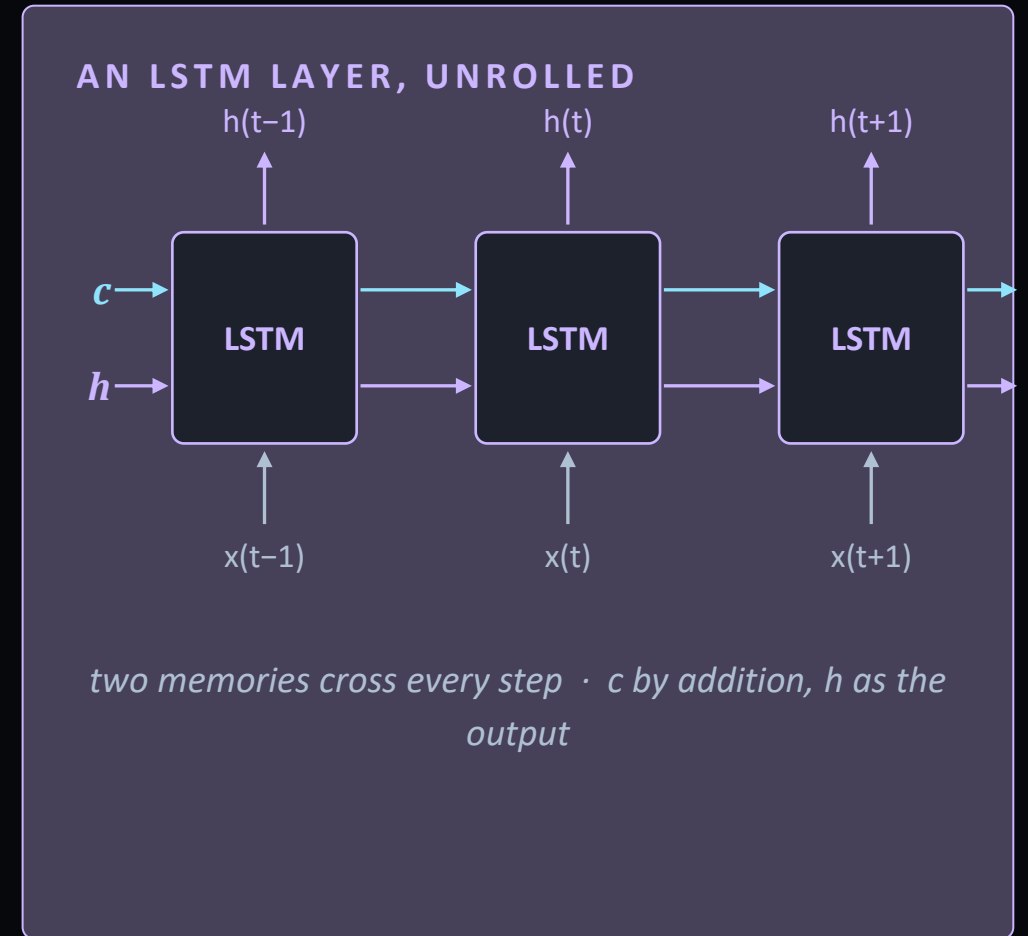
LSTM

KIND, FUNCTION, ARCHITECTURE, APPLICATIONS

- **kind** — a recurrent network with a second, protected memory
- **function** — a sequence in, an output per step or at the end
- **architecture** — a cell state and three gates that control it
- **applications** — forecasting, speech, text, state estimation

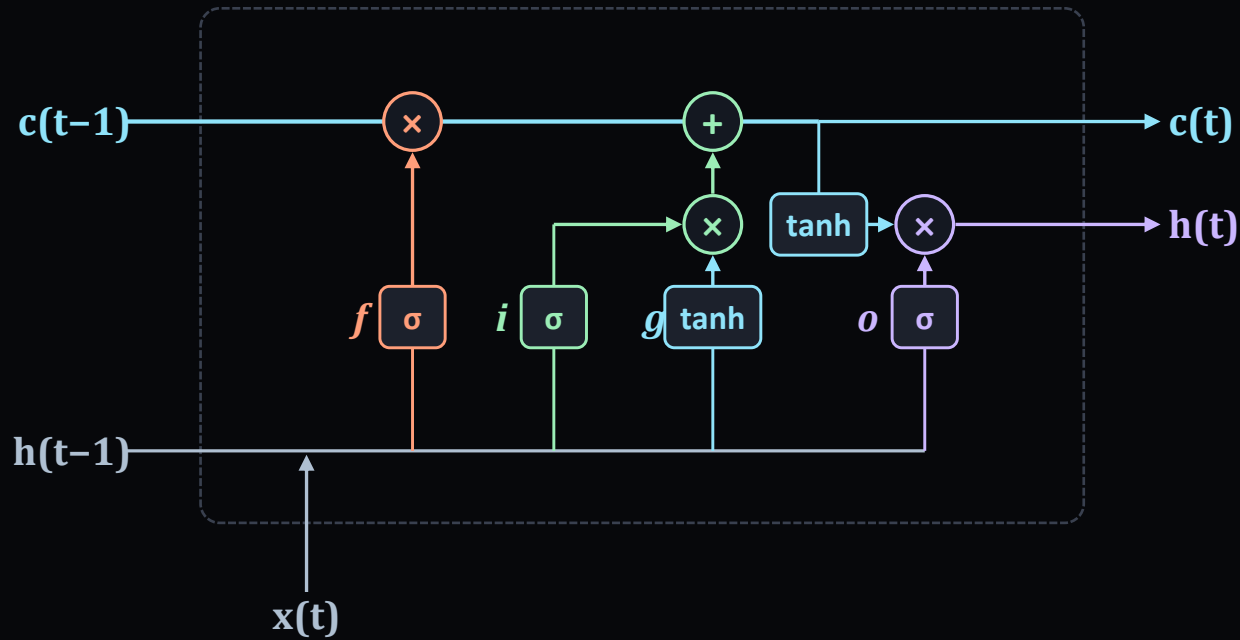
AGAINST THE PLAIN RECURRENCE

	plain recurrence	LSTM
memory	one vector h , overwritten	h , and a cell state c beside it
update	a matrix and a tanh	an addition on c , set by gates
gradient in time	vanishes	survives along c
reach	about ten steps	hundreds of steps
parameters	one set	four sets, four times as many



LSTM Cell

A BELT, AND THREE GATES THAT CONTROL IT



$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

c_t the cell state, f_t the forget gate, i_t the input gate, g_t the candidate, $\odot$ elementwise

the belt is an addition, so c passes through: a residual connection in time

THE SIX EQUATIONS, GATE BY GATE

f $f_t = \sigma(W_f[x_t, h_{t-1}] + b_f)$

forget gate — how much of the belt to keep

i $i_t = \sigma(W_i[x_t, h_{t-1}] + b_i)$

input gate — how much new to add

g $g_t = \tanh(W_g[x_t, h_{t-1}] + b_g)$

candidate — new content from x and h , not x itself

o $o_t = \sigma(W_o[x_t, h_{t-1}] + b_o)$

output gate — how much to show as h

c $c_t = f_t \odot c_{t-1} + i_t \odot g_t$

cell update — the line drawn on the left

h $h_t = o_t \odot \tanh(c_t)$

hidden state — the output

σ : sigmoid, 0 to 1 — a valve · $\tanh$: content, -1 to 1

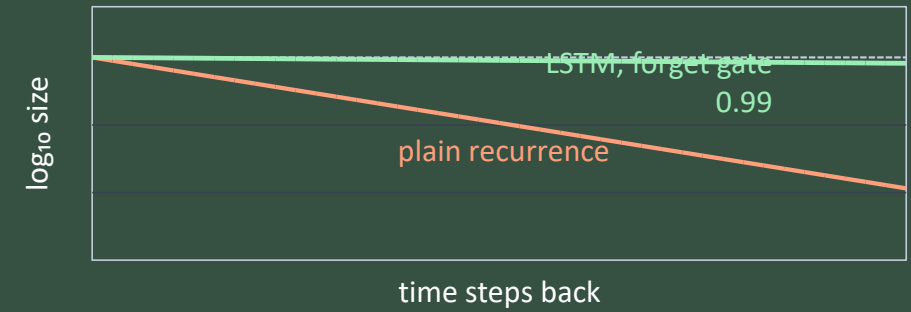
Using an LSTM

SIZE, INITIALISE, AND KNOW THE COST

- **hidden width** — start at 32, grow only if it underfits
- **layers** — one, or two at most
- **forget bias at one** — it remembers by default
- **clip the gradient** — always
- **only offline** — read backwards too, never on a live signal

DECIDE · 2 MIN In your own words, what does a memory give a forecaster that a fixed window does not? Then, in the table on the right, the LSTM matches the perceptron's error with three times the parameters — for a substation controller, which would you deploy?

GRADIENT BACK IN TIME



MEASURED IN NOTEBOOK 04

model	parameters	held-out MSE
plain recurrent	305	0.00127
LSTM	1,169	0.00079
perceptron	417	0.00079

Self-Attention

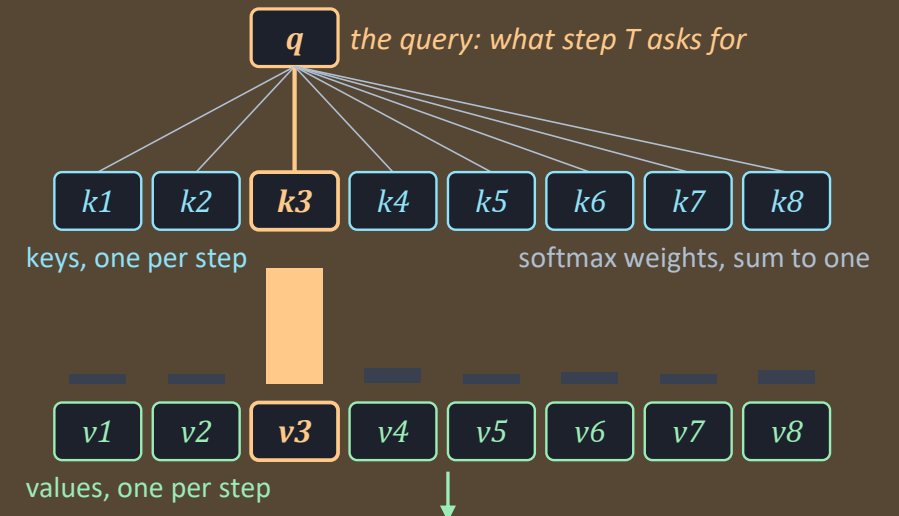
KIND, FUNCTION, ARCHITECTURE, APPLICATIONS

- **kind** — a layer in which every step reads every other step
- **function** — a weighted average, with weights from the data
- **architecture** — query, key, value; a softmax; a weighted sum
- **applications** — long load forecasts, fault records, language

AGAINST A DENSE AND A RECURRENT NETWORK

	dense	recurrent	attention
input	fixed length	a sequence, in order	a set, any size
the past	not kept	carried in h	read directly
step 1 to step T	—	T – 1 steps	one step
weights	fixed	fixed, shared	depend on the input
cost	small	grows with T	grows with T ²

ONE LOOKUP



the output: $\sum \text{weight} \times \text{value}$ — here, mostly v3

no memory to carry: the whole window is in the input at once

every step asks · every step answers · in one parallel operation

Attention in Matrix Form

ALL STEPS AT ONCE, AS MATRICES

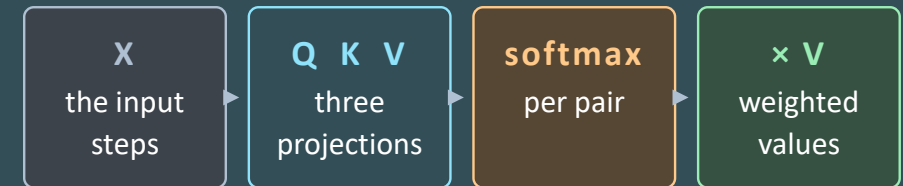
- **three projections** — queries Q , keys K , values V from the input X
- **scores** — every query against every key, then scaled down
- **softmax** — each row becomes weights that sum to one
- **output** — the weights times the values

$$Q = XW_q \quad K = XW_k \quad V = XW_v$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

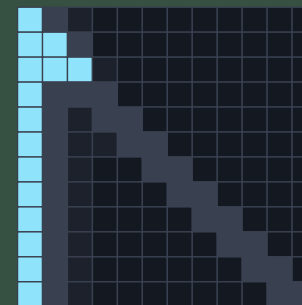
Q, K, V queries, keys and values, d the width of one head, the weights sum to 1

THE STEPS, AS MATRICES



one matrix product for all steps together

WHAT NOTEBOOK 04 LEARNED



rows: each step
columns: the steps it reads
bright: heavy weight

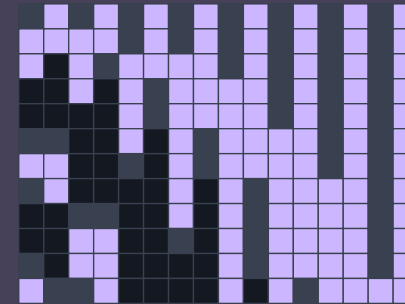
left: the same hour yesterday
diagonal: the last few hours

Positional Encoding

TELLING ATTENTION WHERE EACH STEP IS

- **attention ignores order** — shuffle the steps, same answer
- **on a graph that was the point** — in time it is a defect
- **the fix** — add a vector to each step that says where it is
- **fixed or learned** — notebook 04 learns one per position
- **the recurrent network needs none** — order is built in

ONE VECTOR PER POSITION



rows: positions 1 to 12
columns: the dimensions

fast waves on the left,
slow on the right:
every row is different

THE SAME PROPERTY, OPPOSITE SIGN

graph network ignoring the numbering: right

time series ignoring the order: wrong

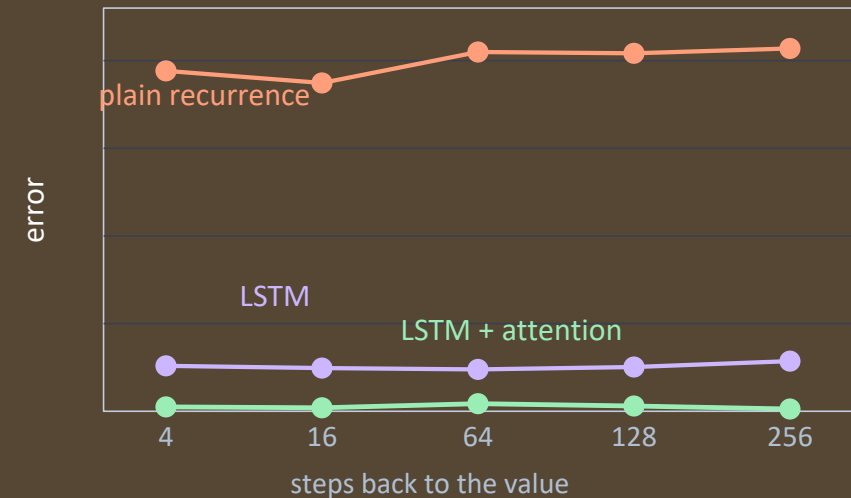
What Attention Adds

A VALUE FAR BACK, CARRIED OR READ

- **the test** — report a value from L steps back
- **recurrence** — must carry it through all L steps
- **attention** — reads that step directly, any L
- **measured** — RNN 1.00, LSTM 0.13, with attention 0.01
- **notebook 04** — 24 hours is short; a perceptron already matches

TALK · 2 MIN In one sentence, what can attention do that recurrence cannot? Then, in the plot on the right, the LSTM's error stays flat and the attention layer's is near zero — what does the LSTM pay at every distance?

ERROR AGAINST THE DISTANCE



1.0 means answering zero · one attention layer: 10 × more accurate than the LSTM

Transformer

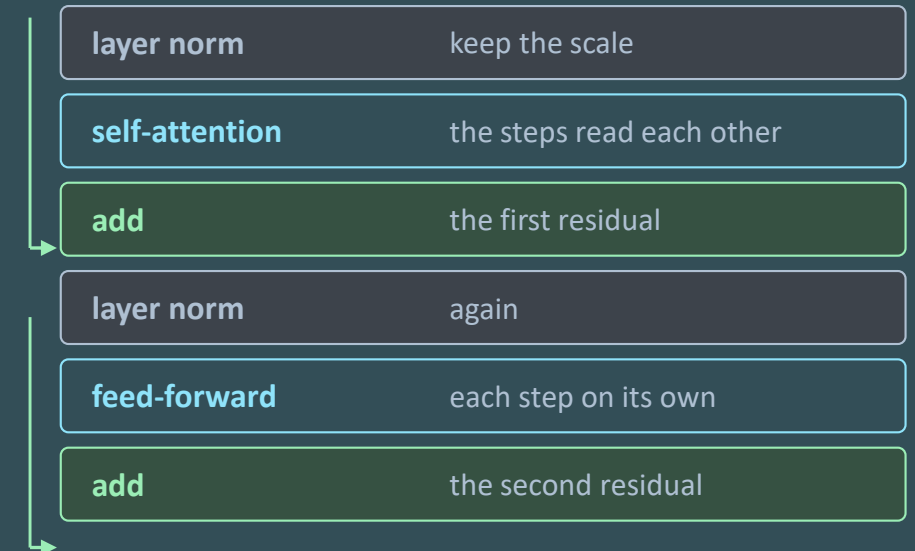
KIND, FUNCTION, ARCHITECTURE, APPLICATIONS

- **kind** — a stack of attention layers, no recurrence
- **function** — a sequence in, a sequence out, all steps at once
- **architecture** — N identical layers, each with two residuals
- **applications** — language models, translation, vision

AGAINST THE RNN AND THE LSTM

	RNN, LSTM	transformer
step 1 to step T	T – 1 steps	one step
computation	one step after another	all steps in parallel
order	built in	added: positional encoding
cost in length	linear	quadratic
depth in practice	one or two layers	six to a hundred layers

ONE LAYER, REPEATED N TIMES



$$\mathbf{X} \leftarrow \mathbf{X} + \text{SA}(\text{LN}(\mathbf{X})) \quad \mathbf{X} \leftarrow \mathbf{X} + \text{FFN}(\text{LN}(\mathbf{X}))$$

SA self-attention, FFN a network per step
LN layer normalisation, $\mathbf{X}$ the steps

no exercise builds one · the two skips are L5.1's

Sequence Models Compared

FOUR MODELS, ONE FORECAST

- **the same task** — the next hour from the last 24
- **every model is better than persistence** — by three to five times
- **the plain RNN is the worst** — its memory fades
- **the LSTM repairs it** — with three times the parameters
- **attention is best on this run** — by a small margin
- **the perceptron is right beside them** — a 24-hour window is short

MEASURED IN NOTEBOOK 04

model	parameters	held-out MSE	vs persistence
persistence	0	0.00332	1.00
plain RNN	305	0.00127	0.38
LSTM	1,169	0.00079	0.24
perceptron	417	0.00079	0.24
attention	409	0.00071	0.21

one run, seed zero · is the lead real? next slide

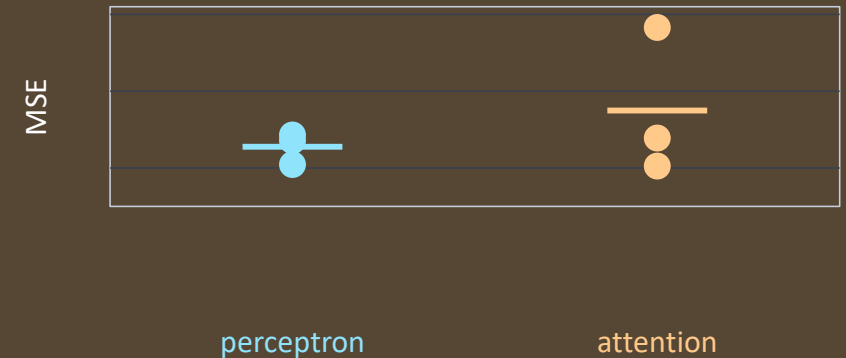
Seed Spread and Residuals

TWO CHECKS BEFORE CALLING A WINNER

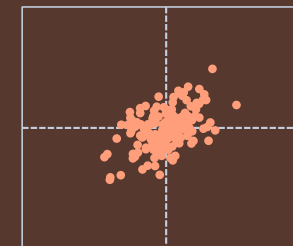
- **seeds** — rerun each model from three random starts
- **the spreads overlap** — attention's lead belonged to the seed
- **residuals** — plot each error against the next one
- **the plot leans** — the errors are still predictable
- **the rule** — a win inside the spread is not a result

TALK · 2 MIN How would you decide that one model is really better than another? Then, in the upper plot, which model would you have reported as better from seed zero alone, and what do the other two seeds do to that claim?

THREE SEEDS EACH



ERROR AT t AGAINST ERROR AT $t + 1$



it leans:
correlation 0.50

the errors are
still predictable

Challenges

SHUFFLING BEFORE SPLITTING

overlapping windows share 23 of 24 hours · split in time order, or the held-out error is fiction

NO POSITIONAL ENCODING

attention cannot tell first from last · it still trains and reports a loss · add the encoding

READING BACKWARDS ON A LIVE SIGNAL

a bidirectional recurrent model reads the future · offline analysis only, never on a controller

SIX CHECKS BEFORE TRUSTING A MODEL

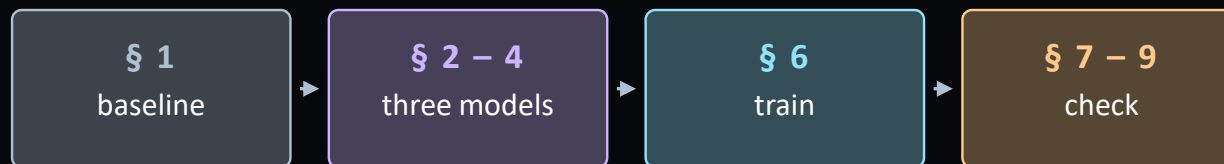
split	in time order
compare	against persistence first
rerun	from three seeds
plot	the residuals against each other
vary	the window length
test	on another site or machine

the first four take ten minutes · they catch most false wins

Exercise 5

NOTEBOOK 04 IS THIS LECTURE'S

- § 1 — baselines first: persistence
- § 2 – 4 — the RNN, the LSTM and attention, written out
- § 6 — train all four, including a perceptron
- § 7 — three seeds: is attention really better?
- § 8 – 9 — what attention reads, and the residuals
- **notebook 05** — the report, for both halves of this block



THE QUESTIONS NOTEBOOK 04 ANSWERS

- § 3 does the LSTM repair the plain RNN, and at what cost?
- § 7 does attention's lead survive three seeds?
- § 9 what do the residuals show that the error did not?

REPORT QUESTION

Four models within a factor of two. Which would you deploy on a substation controller, and which number decided it?

Key Takeaways

1 RNN — a memory, fed back

One cell, shared over time, keeps a hidden state. Its gradient through time shrinks at every step, so it forgets after about ten steps.

2 LSTM — a memory that lasts

A second state, the cell state, updated by addition and controlled by three gates. It reaches hundreds of steps, at four times the parameters.

3 Attention — reading, not carrying

Every step reads every other directly, with weights from the data. One step between any two, all in parallel; order must be added.

4 Transformer — attention stacked

One layer is self-attention and a per-step network, each wrapped in a residual connection. Nothing in this course builds one.

5 Comparing — two checks

In notebook 04 all four models beat persistence. Rerun from three seeds and look at the residuals before calling a winner.

Next — L6.1: loss functions and gradients.

Questions

TEN QUESTIONS FOR THE ORAL EXAMINATION

- 1 What kind of network is a recurrent network, and what function does it implement on a sequence? *GBC ch. 10.1 – 10.2*
- 2 Describe its architecture: the hidden state, the unrolled network, and why the gradient over time vanishes? *GBC ch. 10.2, 10.7*
- 3 What kind of network is an LSTM, and how does it differ from the plain recurrence in memory, update and reach? *GBC ch. 10.10*
- 4 Describe the LSTM cell: the cell state, the three gates, the candidate, and the six equations? *GBC ch. 10.10; Ex_05 notebook 04*
- 5 How is an LSTM sized and initialised, and why set the forget bias to one? *GBC ch. 10.10 – 10.11; Ex_05 notebook 04*
- 6 What kind of layer is self-attention, and what do queries, keys and values do? *UDL ch. 12.2*
- 7 Write attention in matrix form, and explain why a positional encoding is needed? *UDL ch. 12.2 – 12.3; Ex_05 notebook 04*
- 8 How does attention differ from recurrence and from a dense layer, and what did adding one layer measure to? *UDL ch. 12.1; tools/deck/make_attention_fig.py*
- 9 Describe one transformer layer: what two blocks does it stack, and where are its residual connections? *UDL ch. 12.4*
- 10 A model wins by ten per cent on one run. What do you check first, and what do the residuals add? *Ex_05 notebook 04, sections 7 and 9*

NEXT

L6.1 — Loss Functions and Gradients

Every architecture this course uses is now in hand: dense, convolutional, residual, graph, recurrent and attention. Next: how they are trained.

BEFORE THE NEXT SESSION

read	UDL ch. 5.1 – 5.3
run	Ex_05 notebooks 04 and 05
bring	your three-seed spread
think about	which model to deploy, and why

bring your three-seed numbers from notebook 04